

Algorithmic Roster Construction: Strategic Talent Acquisition under Competitive Scarcity

A Fantasy Baseball Testbed for Optimization and Real-Time Heuristics*

OR114-2 Final Project, Group 4

B13705051 周孟承 B13902103 鄭宇宏 B13303153 詹舒宇 B11705039 李盈盈

1 Introduction: The General Manager's Dilemma

As the 2026 MLB season approaches, front offices across the league are focused on one critical objective: the championship. The cornerstone of a winning season is not just how the players perform on the field, but how they are selected during the off-season draft. Developing a "winning strategy" to build an elite team is the primary concern of professional General Managers and the obsession of millions of fans participating in online simulations such as Fantasy Baseball. Regardless of the platform, the core problem remains the same: how does a manager use a limited sequence of picks to secure the highest possible utility before opponents deplete the talent pool?

The draft represents a critical portfolio optimization puzzle for modern sports roster construction. In an era where scouting analytics and player evaluations have largely converged, the competitive edge has shifted from merely identifying talent to the strategy of execution. This project models the draft process as a sequential resource acquisition problem, using Fantasy Baseball as a high-fidelity competitive testbed to study how decision-makers can maximize objective value under strict structural constraints and intense market competition.

The motivation for this study stems from the inherent complexity of a draft environment, which forces managers to confront a fundamental "Timing Paradox" —the challenge of securing a specific asset now before market timing and positional scarcity cause equivalent options to disappear. This leads to difficult strategic questions: Is it mathematically superior to draft a "balanced" team of solid contributors, or a "M-shaped" roster where elite superstars are paired with low-cost fillers?

Furthermore, the problem is compounded by the threat of market disruption and the sheer scale of the decision space. Even with a "perfect" plan, targets are inevitably "sniped" by opponents in a live environment, forcing a manager to pivot instantly. In a standard league, a team must fill 16 unique slots while competing against 12 other teams for a pool of over 1,000 eligible players. The resulting combinatorial possibilities are astronomical. Because real-world drafts are governed by strict timers, a human manager cannot manually flip through a binder of hundreds of variations (from Plan A to Z alternatives) to find an optimal response. The practical decision is not simply selecting the player with the highest isolated score (the naive "Best Player Available" approach), but deciding how to trade off immediate value, positional scarcity, and future availability: A high-projection player might be suboptimal if he occupies a position where supply is abundant, whereas a lower-projection player might be highly valuable if he fills a scarce roster slot. Consequently, there is a significant need for a decision-support framework that provides a deployable balance between theoretical optimality and real-time execution speed. This project then aims to bridge that gap by developing a scalable optimization strategy capable of building an elite team under pressure.

*Source code and data: https://github.com/089487/OR_Final_Project

2 Problem Description

While the actual MLB draft uses a fixed order based on reverse standings, we abstract our environment into a **Snake Draft**, serving for a highly competitive, fair, and strategically demanding sequence of decisions. In a snake draft, a manager owns a predetermined sequence of picks, and the player pool is continually depleted by opponents. At each owned pick, the manager must choose exactly 1 available player from the remaining pool. Every player in the pool possesses 3 attributes:

1. **Projected Value:** The expected statistical output (points) for the season.
2. **Eligible Positions:** The real-life positions the player can occupy.
3. **Average Draft Position (ADP):** A market consensus of when the player is expected to be drafted by opponents [2].

The objective is to maximize the sum of projected points for the starting roster. The drafted players must perfectly fulfill strict positional requirements. The base roster structure used in our study demands 16 active slots, as detailed in Table 1. (Note: The Utility (**Util**) slot is hitter-only, meaning pitchers cannot be assigned there).

Table 1: Base fantasy baseball roster requirements.

Position	C	1B	2B	3B	SS	OF	Util	SP	RP
Required Slots	1	1	1	1	1	3	1	5	2

A player can only be drafted if he is still available in the market. We treat ADP as a proxy for market depletion using FantasyPros consensus ADP [2]. If the GM’s current draft pick occurs much later than a player’s ADP, he is considered successfully drafted by an opponent and is unavailable. By integrating ADP market constraints and positional flexibility into the sequence of decisions, draft planning is transformed from a subjective ranking exercise into a strict operations research problem.

3 Model Formulation

We then first formulate the exact Integer Programming model to serve as our absolute benchmark.

Let I be the set of players, P the set of roster positions, and K the set of owned draft picks. Each player $i \in I$ has projected points v_i , market expectation adp_i , and eligible positions $E_i \subseteq P$. Each position $p \in P$ requires r_p players. Let S_k be the overall draft number for the manager’s k -th pick.

Decision Variables:

- $y_i \in \{0, 1\}$: 1 if player i is drafted, 0 otherwise.
- $x_{ip} \in \{0, 1\}$: 1 if player i is assigned to position p , 0 otherwise.
- $z_{ik} \in \{0, 1\}$: 1 if player i is selected precisely at owned pick k , 0 otherwise.

Objective Function: Maximize total roster points.

$$\max \sum_{i \in I} v_i y_i$$

Constraints:

$$\sum_{i \in I} z_{ik} = 1, \quad \forall k \in K \quad (1)$$

$$\sum_{k \in K} z_{ik} = y_i, \quad \forall i \in I \quad (2)$$

$$\sum_{p \in P} x_{ip} = y_i, \quad \forall i \in I \quad (3)$$

$$\sum_{i \in I} x_{ip} = r_p, \quad \forall p \in P \quad (4)$$

$$z_{ik} = 0 \quad \text{if } S_k > \text{adp}_i + \delta, \quad \forall i \in I, k \in K \quad (5)$$

$$x_{ip} = 0 \quad \text{if } p \notin E_i, \quad \forall i \in I, p \in P \quad (6)$$

$$y_i, x_{ip}, z_{ik} \in \{0, 1\} \quad \forall i, p, k \quad (7)$$

Constraint (1) enforces exactly one selection per pick. (2) and (3) ensure logical consistency between picking, drafting, and assigning. (4) dictates the roster shape. Constraint (5) is the vital **Market Availability Constraint**: a player cannot be drafted if the current pick S_k is later than his expected depletion point (ADP + a tolerance buffer δ).

4 Algorithms

While the IP provides mathematical optimum, solving it is NP-Hard. Because baseball players don't need to actively register for the draft (anyone meeting age/school requirements is eligible), the actual pool scales to tens of thousands of players. When the problem scale rises, the branch-and-bound tree exceeds reasonable time limits or crashes completely due to exponential complexity. Therefore, we developed two heuristic algorithms using Heaps to achieve extreme computational efficiency.

4.1 Direct Greedy (Baseline DG)

The Direct Greedy algorithm (DG) serves as our baseline heuristic, representing the standard “fill the biggest need with the best available player” mentality. At each pick, DG scans all open positions, identifies the position with highest scarcity ($\frac{\text{Remaining Need}}{\text{Available Players}}$), and selects the player with the highest projected points for that position.

Algorithm 1 Direct Greedy (DG)

```

1: Initialize Max-Heaps for each position  $p \in P$  based on  $v_i$ .
2: Sort all players by  $(\text{adp}_i + \delta)$  to create an expiration_order.
3: for each owned pick  $k \in K$  do
4:   Pop expired players from heaps where  $\text{adp}_i + \delta < S_k$ 
5:   best_position  $\leftarrow$  null, max_scarcity  $\leftarrow -\infty$ 
6:   for each position  $p$  with  $r_p > 0$  do
7:     Clean heap (remove drafted/expired players lazily)
8:     scarcity  $\leftarrow \frac{\text{Remaining Need for } p}{\text{Active Eligible Players for } p}$ 
9:     if scarcity  $>$  max_scarcity (tie-break by top player's  $v_i$ ) then
10:      best_position  $\leftarrow p$ 
11:      max_scarcity  $\leftarrow$  scarcity
12:     end if
13:   end for
14:   Draft top player from best_position heap and update remaining needs.
15: end for

```

4.2 Opportunity Cost Greedy (Proposed Method)

Opportunity Cost Greedy (OCG) is our flagship algorithm. Instead of only looking at the *current* turn, OCG looks ahead to the *next* owned pick to avoid being myopic. It calculates the **Opportunity Cost** of waiting for a position by peeking into the future availability of players, subtracting the points of the best player available *next round* from the best player available now.

Algorithm 2 Opportunity Cost Greedy (OCG)

```

1: Initialize two sets of Max-Heaps for each position  $p$ : current_heaps and future_heaps.
2: for each owned pick  $k \in K$  (let next pick be  $k_{next}$ ) do
3:   Expire players in current_heaps based on  $S_k$ 
4:   Expire players in future_heaps based on  $S_{k_{next}}$ 
5:    $best\_choice \leftarrow \text{null}$ ,  $max\_priority \leftarrow (-\infty, -\infty, -\infty)$ 
6:   for each position  $p$  with  $r_p > 0$  do
7:      $v_{cur} \leftarrow$  points of top player in current_heaps[ $p$ ]
8:      $v_{fut} \leftarrow$  points of top player in future_heaps[ $p$ ]
9:      $score \leftarrow v_{cur} - v_{fut}$  ▷ Delay Cost Calculation
10:     $forced \leftarrow \text{True}$  if (Active Players  $\leq$  Need)
11:     $priority \leftarrow (forced, score, v_{cur})$ 
12:    if  $priority > max\_priority$  then
13:       $max\_priority \leftarrow priority$ 
14:       $best\_choice \leftarrow (\text{Top Player}, p)$ 
15:    end if
16:  end for
17:  Draft player from  $best\_choice$  and update remaining needs.
18: end for

```

4.3 Time Complexity Analysis

Let n be the total number of players in the draft pool, r be the number of roster slots (total picks per manager), and p be the number of distinct positions.

- **Initialization:** Building the initial max-heaps and sorting the expiration list takes $O(n \log n)$.
- **Drafting Loop:** Checking the top of a heap is $O(1)$. For each of the r picks, the algorithm scans p positions, given the time complexity of $O(r \cdot p)$.
- **Heap Cleaning (Lazy Deletion):** Expiring drafted players is done by popping invalid elements at the top of the heap. Across the *entire draft process*, a player is popped from a position heap at most once. Therefore, the total time spent cleaning heaps is bounded by $O(n \log n)$.

Total Time Complexity for both DG and OCG is $O(n \log n + r \cdot p)$. Because r and p are tiny constants relative to n , the algorithm runs in near-linear time relative to sorting the player pool. This mathematically guarantees extreme scalability compared to the exponential worst-case nature of IP.

5 Data Collection and Generation

To evaluate our algorithms, we utilize both real-world and a highly customizable Synthetic Data.

5.1 Real Data Collection

Real data validation serves as our first step of evaluation. We synthesize our dataset from FantasyPros, utilizing **projected points as a player utility proxy** (v_i) [1] and **Average Draft Position (ADP)**

as a market availability window proxy [2]. Figures 1 and 2 display examples of the underlying data structure.

PLAYER	AB	R	HR	RBI	SB	AVG	OBP	H	2B	3B	BB	SO	SLG	OPS	ROST%
Shohei Ohtani (LAD - SP,DH)	572	124	47	111	24	.283	.382	162	28	5	92	164	.598	.980	100%
Aaron Judge (NYY - LF,CF,RF,DH)	520	110	46	114	9	.292	.417	152	26	1	112	160	.609	1.026	99%
Bobby Witt Jr. (KC - SS)	603	101	29	93	34	.292	.345	176	38	6	49	114	.518	.863	99%

Figure 1: Example of 2026 player projected batting and pitching statistics from FantasyPros [1].

Import a Team

2026

Rank	Player (Team)	Yahoo	CBS	RTS	NFBC	FT	ESPN	AVG
1	Shohei Ohtani (LAD - SP,DH)		1	2	1	1	1	1.0
2	Aaron Judge (NYY - LF,CF,RF,DH)	1	2	1	2	2	2	1.8
3	Shohei Ohtani (Batter) (LAD - DH)	2						2.0
4	Bobby Witt Jr. (KC - SS)	3	3	4	3	4	4	3.6
5	Juan Soto (NYM - LF,RF,DH)	4	4	3	4	3	3	3.6

Figure 2: Example of player Average Draft Position (ADP) from FantasyPros consensus ADP [2].

Player values are calculated using two distinct standard scoring benchmarks: Yahoo (which focuses on traditional counting stats) [3] and FanGraphs (incorporates more advanced player metrics based on linear weights) [4]. The detailed weights for both batters and pitchers are outlined in Table 2:

Table 2: Scoring weights for Yahoo and FanGraphs/Ottoneu formats [3], [4].

Metric	Batters		Metric	Pitchers	
	Yahoo	FanGraphs		Yahoo	FanGraphs
1B (or H)	2.6	5.6	IP	3.0	7.4
2B	5.2	2.9	K	3.0	2.0
3B	7.8	5.7	W	8.0	—
HR	10.4	9.4	SV	8.0	5.0
R	1.9	—	ER	-3.0	—
RBI	1.9	—	BB	-1.3	-3.0
BB	2.6	3.0	H	-1.3	-2.6
SB	4.2	1.9	HBP	-1.3	-3.0
HBP	0 (2.6)	0 (3.0)	HR	—	-12.3
AB	—	-1.0	HLD	—	4.0
CS	—	0 (-2.8)			

Slight modifications were made to the original projected stats to align with these scoring categories [3], [4]. For example, since Hit By Pitch (HBP) and Caught Stealing (CS) for batters are lacked in the raw stats, we effectively set their scoring weights to zero (original standard weights are shown in parentheses in the table).

Additionally, across all experiments (both real and synthetic data), the manager’s initial draft position is standardized to the middle of the draft order (e.g., the 6th pick in a 12-team league) to maintain a consistent evaluation baseline.

5.2 Experimental Synthetic Data Generation

While real data confirms practical viability, a further validation is performed on synthetic data. The generated data is indispensable as it allows us to mathematically isolate specific operational variables (Factors), such as talent distribution, positional versatility, and market supply; moreover, stress-test the algorithms under controlled extreme conditions.

The total number of generated players in the environment is defined dynamically based on the league size (T), base roster slots ($R = 16$), roster scale multiplier (S), and player-demand ratio (D):

$$N_{\text{players}} = S \times 16 \times T \times D$$

Experimental framework generates synthetic profiles encompassing 4 primary operational factors:

5.3 Factor A: Points Distribution (v_i)

We designed three points distributions to simulate different talent pools, clipping all generated values to a realistic range of $[10.0, 800.0]$ according to the FantasyPros projection data collected above [1]:

- **Normal (Baseline):** Values follow a normal distribution $v_i \sim \mathcal{N}(400, 120^2)$. This represents an average draft class where the majority of players are solid, mid-tier contributors.
- **Uniform:** Values are uniformly distributed $v_i \sim \mathcal{U}(10, 800)$. This flattens the talent curve, offering many viable replacement options and minimizing scarcity.
- **High-Low (Star-Heavy):** We simulate a highly skewed draft class. Only 10% of players are classified as elites, with values drawn from $\mathcal{U}(500, 800)$. The remaining 90% fall into $\mathcal{U}(10, 500)$. In this scenario, missing out on top-tier talent may be severely punished in the objective function.

5.4 Factor B: Position Eligibility Distribution (E_i)

We control how versatile the player pool is using four distinct generation logics:

- **Roster-Ratio (Baseline):** Players are assigned exactly one position, but the probability of generating a specific position aligns precisely with roster demand (e.g., more OF and SP players are generated since there shall be 3 such positioned-players per team).
- **Uniform-by-Type:** A player is randomly designated as a hitter or pitcher, then uniformly assigned a position pattern (including dual-eligibility like 1B;3B or SP;RP).
- **Point-Flexible:** The generator ranks all players by points. Players in the top 20% (elite) have an 80% chance of gaining multi-position eligibility (up to 3 positions). This mimics real life where superstars often possess high versatility.
- **Single-Position:** Every player possesses exactly 1 eligible position. This eliminates all flexibility, representing the ultimate test of scarcity, making early greedy mistakes almost impossible to repair.

5.5 Factor C: Market Conditions and ADP Uncertainty (σ_{adp}, δ)

To generate ADP, we first sort all players by their true projected points in descending order to establish a “True Rank”. We then simulate market noise by adding a Gaussian error term:

$$\text{ADP}_i = \text{TrueRank}_i + \mathcal{N}(0, \sigma_{adp}^2, \delta) \quad (8)$$

We evaluate two specific dimensions of market uncertainty:

- **Market Volatility (σ_{adp}):** We vary σ_{adp} from 0, 30, to 60. A value of $\sigma_{adp} = 0$ represents a perfectly efficient market where opponents draft strictly according to player value. Higher values of σ_{adp} simulate “chaotic” markets where opponents may make unpredictable “reaches” for low-value players or allow elite assets to fall well past their expected rank. (fluctuated around $30/12 \approx 3$ and $60/12 = 5$ rounds)
- **Availability Tolerance (δ):** This parameter represents the manager’s risk threshold within the availability constraint ($S_k \leq ADP_i + \delta$). We test values of $\delta \in \{0, 5, 10\}$. A negative δ represents a conservative, “safety-first” drafting strategy, while a positive δ allows the algorithm to be more aggressive, assuming players will remain available longer than the market average.

Testing under these conditions allows us to verify if the Opportunity Cost Greedy (OCG) algorithm can maintain its strategic edge even when the underlying market data is noisy or biased.

5.6 Factor D: Player Demand Ratio (D)

To test the algorithms under varying levels of scarcity, we manipulate the Player Demand Ratio (D). The factor controls the supply-and-demand economics of the draft pool by dictating the ratio of “Available Players” versus “Total League Demand”. Let T be the number of teams and $R = 16$ be the base roster slots, the total number of generated players is strictly governed by $N_{\text{players}} = R \times T \times D$.

- $D = 1$ (**Tight Market**): The league will draft almost every available player. One greedy mistake means replacing a starter with absolute zero value.
- $D = 3$ (**Baseline Supply**): Representing standard draft conditions with a typical buffer of free undrafted agents.
- $D = 10$ (**High Abundance**): The player pool is a massive ocean of options.

5.7 Scenario Matrix

We then developed 12 distinct scenarios to isolate the impact of each factor. As shown in Table 3, **Scenario S1** serves as the baseline for all comparative analyses. The remaining scenarios follow the factor order introduced above: projected points distribution, position eligibility distribution, market conditions, and player demand ratio. For every scenario, we generate **10 random instances** to calculate the mean (μ) and standard deviation (σ) of the performance metrics.

Table 3: Experimental Scenario Matrix for Draft Optimization

ID	Main Factor	Demand Ratio (D)	Points Distribution (v_i)	Position Eligibility (E_i)	ADP Uncertainty (δ, σ)
S1	Baseline	3	Normal	Roster-Ratio	(0, 30)
S2	Points: Uniform	3	Uniform	Roster-Ratio	(0, 30)
S3	Points: Star-Heavy	3	High-Low	Roster-Ratio	(0, 30)
S4	Pos: Uniform-by-Type	3	Normal	Uniform-by-Type	(0, 30)
S5	Pos: Point-Flexible	3	Normal	Point-Flexible	(0, 30)
S6	Pos: Single-Position	3	Normal	Single-Position	(0, 30)
S7	Market: Efficient	3	Normal	Roster-Ratio	(0, 0)
S8	Market: Mild Noise	3	Normal	Roster-Ratio	(0, 60)
S9	Market: Chaotic	3	Normal	Roster-Ratio	(+5, 30)
S10	Market: Chaotic	3	Normal	Roster-Ratio	(+10, 30)
S11	Demand: High	1	Normal	Roster-Ratio	(0, 30)
S12	Demand: Low	10	Normal	Roster-Ratio	(0, 30)

5.8 Stress Testing Design

To evaluate the absolute computational limits of our algorithms relative to the IP benchmark, we moreover design a separate **Large-Scale Stress Test** outside of the standard factor matrix. Instead of sweeping isolated market parameters, this test scales the raw problem dimensions: Drastically increasing both the available player pool and the roster constraints. The primary objective is to simulate massive, unconstrained draft databases to observe the breaking point of exact optimization (IP) and to validate the asymptotic time efficiency of the OCG heuristic under massive scale.

6 Performance Evaluation

Algorithms are evaluated using two primary metrics across 10 random seeds for each scenario:

- **Optimal Gap Ratio (%)**: The distance from the IP benchmark, defined as $\frac{Z_{IP} - Z_{Heuristic}}{Z_{IP}} \times 100\%$. A lower value indicates higher accuracy.
- **Improvement (%)**: The direct gain of OCG over the DG baseline, calculated per seed as $\frac{Z_{OCG} - Z_{DG}}{Z_{DG}} \times 100\%$ and reported as $\mu \pm \sigma$.

Unless otherwise noted, all synthetic results are reported as Mean $\pm$ Standard Deviation.

6.1 Real-Data Validation

As the first step of validation, we ran the algorithms on the real-world 2026 FantasyPros projections with Yahoo and FanGraphs/Ottoneu scoring rules [1], [3], [4]. As seen in Table 4, OCG successfully trims the point loss caused by naive greedy drafting, confirming its practical viability on real MLB data structures before proceeding to the more comprehensive synthetic validations.

Table 4: Real-data validation (Gap measured in optimal gap ratio versus IP).

Scoring Method	Method	Optimal Gap Ratio
Yahoo 2026	Opportunity Cost Greedy	0.50%
	Direct Greedy	3.24%
FanGraphs 2026	Opportunity Cost Greedy	1.24%
	Direct Greedy	3.36%

6.2 Factor A: Points Distribution

Table 5 summarizes the impact of the talent distribution. Under the **Baseline (Normal)** distribution, OCG achieves a tight gap of 1.92%, reducing the error of the DG baseline by more than half. The advantage is most pronounced in **Star-Heavy** markets (S3), where OCG prevents elite talent from being snatched by opponents, yielding a **6.10%** average improvement in total roster value.

Table 5: Results for Factor A: Points Distribution. (the optimality gap (%) $\pm$ standard deviation.)

Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
Normal (Baseline)	4.79% $\pm$ 1.18%	1.92% $\pm$ 0.57%	3.02% $\pm$ 1.34%
Uniform	3.24% $\pm$ 0.77%	1.44% $\pm$ 0.59%	1.87% $\pm$ 0.65%
High-Low (Star-Heavy)	8.39% $\pm$ 2.40%	2.87% $\pm$ 1.32%	6.10% $\pm$ 3.46%

6.3 Factor B: Position Eligibility Distribution

Table 6 illustrates performance across different positional flexibility environments. Compared to the **Baseline (Roster-Ratio)**, *point-flexible* is the most forgiving environment (gap drops to 1.34%) because top players can seamlessly plug holes in the roster. Conversely, *single-position* creates a rigid, unforgiving draft. Even when mistakes are impossible to patch with utility players, OCG dominates Direct Greedy (0.94% vs 5.11%), proving that looking one pick ahead prevents catastrophic positional deficits later in the draft.

Table 6: Results for Factor B: Position Eligibility Distribution.

Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
Roster-Ratio (Baseline)	4.79% $\pm$ 1.18%	1.92% $\pm$ 0.57%	3.02% $\pm$ 1.34%
Uniform-by-Type	4.89% $\pm$ 0.62%	1.07% $\pm$ 0.43%	4.02% $\pm$ 0.82%
Point-Flexible	5.36% $\pm$ 1.66%	1.34% $\pm$ 0.68%	4.27% $\pm$ 1.83%
Single-Position	5.11% $\pm$ 1.48%	0.94% $\pm$ 0.65%	4.42% $\pm$ 1.95%

6.4 Factor C: Market Conditions and ADP Uncertainty

Table 7 demonstrates performance under chaotic draft markets. We split the analysis into two sweeps: varying market noise (σ_{adp}) and varying availability tolerance (δ).

When the market is perfectly efficient ($\sigma_{\text{adp}} = 0$), predicting opponent behavior is trivial, and both algorithms perform exceptionally well. As noise increases to $\sigma_{\text{adp}} = 60$, the draft becomes unpredictable. Direct Greedy struggles significantly (gap over 5%), whereas OCG remains highly robust (2.43%). Similarly, OCG consistently keeps the gap under 2% regardless of the availability tolerance (δ). Because OCG bases its priority on the *relative difference* (Opportunity Cost) rather than absolute values, it inherently hedges against market volatility.

Table 7: Factor C: Market Conditions and ADP Uncertainty results. The first sweep tests market noise (σ_{adp}) with fixed $\delta = 0$. The second tests nonnegative tolerance (δ) with fixed $\sigma_{\text{adp}} = 30$. Values represent the optimality gap (%) $\pm$ standard deviation.

Sweep	Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
Noise	$\sigma_{\text{adp}} = 0$ (Efficient)	1.47% $\pm$ 0.48%	0.48% $\pm$ 0.23%	1.00% $\pm$ 0.50%
	$\sigma_{\text{adp}} = 30$ (Moderate)	4.79% $\pm$ 1.18%	1.92% $\pm$ 0.57%	3.02% $\pm$ 1.34%
	$\sigma_{\text{adp}} = 60$ (Chaotic)	5.56% $\pm$ 1.58%	2.43% $\pm$ 1.55%	3.33% $\pm$ 1.43%
Tolerance	$\delta = 0$ (Baseline)	4.79% $\pm$ 1.18%	1.92% $\pm$ 0.57%	3.02% $\pm$ 1.34%
	$\delta = +5$	5.54% $\pm$ 1.57%	1.64% $\pm$ 0.75%	4.16% $\pm$ 1.50%
	$\delta = +10$ (Permissive)	5.42% $\pm$ 1.21%	1.72% $\pm$ 0.95%	3.92% $\pm$ 1.03%

6.5 Factor D: Player Demand Ratio

Table 8 highlights scaling under tight vs. loose supply (Player Demand Ratio D). When supply is brutally tight ($D = 1$, meaning almost every player must be drafted), Direct Greedy collapses (8.81% gap) as it blindly paints itself into a corner. OCG, aware of the impending scarcity, navigates the tight market with only a 2.64% gap. In a high-abundance scenario ($D = 10$), OCG drops the error rate to a mere 1.16%.

Table 8: Factor D: Player Demand Ratio results (Fixed Roster Scale = 1).

Level	DG Gap (%)	OCG Gap (%)	Improvement (%)
D = 3 (Baseline)	4.79% $\pm$ 1.18%	1.92% $\pm$ 0.57%	3.02% $\pm$ 1.34%
D = 1 (Demand: High)	8.81% $\pm$ 2.19%	2.64% $\pm$ 1.23%	6.82% $\pm$ 3.13%
D = 10 (High Abundance)	3.29% $\pm$ 0.85%	1.16% $\pm$ 0.51%	2.20% $\pm$ 0.83%

6.6 Large-Scale Stress Testing

While the IP finds perfect rosters in smaller instances, what happens when the scouting department dumps a universe of hundreds of thousands of eligible players into the database? This presents the most critical argument for our front office pitch: **Scalability**.

Table 9 showcases the **Large-Scale Stress Test**. To understand the complexity of the IP, we approximate its search space by the variable count. For n available players, p roster positions, and a roster size of r , the approximate number of decision variables is:

$$\text{Approx. Variables} = n \times (1 + p + r)$$

The Branch-and-Bound tree of the IP explores an exponentially growing space, and as we scale the roster size to 256 and the player pool to 184,320, the approximate variable count reaches nearly 50 million. Consequently, the IP runtime explodes and eventually hits our predefined 30-minute (1800 seconds) solver time limit for the largest target, triggering a **TIMEOUT_ERROR**. In stark contrast, confirming our $O(n \log n)$ complexity analysis, **Opportunity Cost Greedy processes the massive 50-million variable equivalent environment in just 23.286 seconds**.

Table 9: Stress Test: Execution Times for DG, OCG, and IP.

Level	Players	Teams	Picks	DG / OCG / IP Time(s)	DG Gap (%)	OCG Gap (%)
Small	5,760	12	48	0.790 / 0.852 / 4.235	2.430%	1.098%
Medium	9,600	15	64	0.951 / 1.144 / 12.230	1.771%	0.646%
Large	21,600	15	96	1.981 / 2.344 / 54.267	0.940%	0.407%
XLarge	64,000	20	160	6.172 / 10.360 / 276.895	0.270%	0.137%
Target	184,320	24	256	17.760 / 23.286 / TIMEOUT	—	—

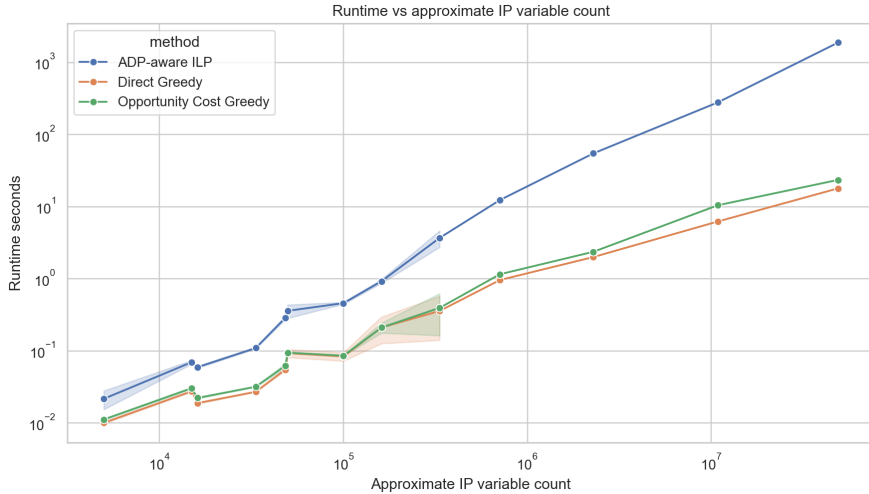


Figure 3: Runtime comparison for Stress Tests. IP scales exponentially while heuristics scale linearly.

6.7 Strategic Insights and Competitive Edge

After evaluating the algorithm across 12 distinct scenarios, we identify the conditions where the **Opportunity Cost Greedy (OCG)** algorithm provides the most significant strategic advantage. Table 10 highlights the top-performing scenarios based on the improvement over the baseline.

Table 10: Key Strategic Scenarios: Maximum Improvement of OCG over DG.

Factor	Scenario ID	Environment Condition	Improvement (%)
Points Distribution (Factor A)	S3	Points: Star-Heavy	+6.10%
Position Eligibility (Factor B)	S6	Pos: Single-Position	+4.42%
Market Conditions (Factor C)	S9	Market: Chaotic ($\delta = +5$)	+4.16%
Player Demand Ratio (Factor D)	S11	Demand: High ($D = 1$)	+6.82%

Analysis of the Competitive Edge: The highlighted scenarios reveal where cost-of-delay reasoning matters most. OCG secures elite assets before the value cliff occurs (S3), prevents early positional mistakes that cannot be repaired later under single-position eligibility (S6), remains robust to uncertain and chaotic opponent behavior (S9), lastly, avoids “positional blindness” happened in DGs when facing tight markets (S11).

6.8 Competitive Market Simulation

To evaluate OCG in a multi-agent environment, we implemented a 12-team draft simulation. Unlike the previous IP-benchmark experiments, this test measures how one OCG-managed team performs while competing directly against 11 DG-managed teams for the same player pool.

Methodology: Using real-world 2026 FantasyPros projections, FantasyPros ADP, and Yahoo/FanGraphs scoring rules, we ran 12 separate snake drafts and shifted the OCG team’s starting position from the 1st pick to the 12th pick [1], [2], [3], [4]. All teams followed the same roster requirements and market availability rules.

Results and Analysis: Figure 4 compares the OCG team’s objective and rank against DG opponents across all draft positions. OCG achieves first place across all Yahoo positions and first or second place across all FanGraphs positions. This suggests that cost-of-delay reasoning remains robust even when multiple agents compete for the same scarce resources.

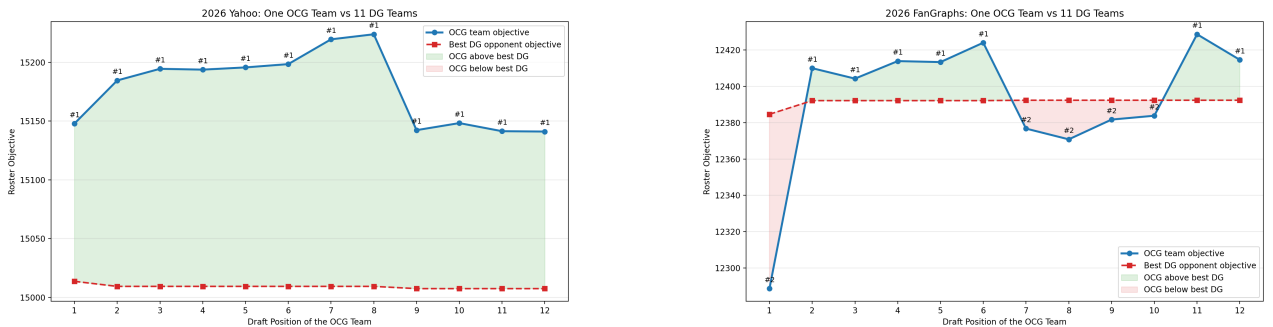


Figure 4: Competitive simulation: one OCG team vs. 11 DG teams across all draft positions. Left: Yahoo; right: FanGraphs. OCG ranks first for Yahoo and either first or second for FanGraphs.

7 Conclusions and Strategic Outlook

The results of this project demonstrate that roster construction in competitive markets is a complex sequential decision problem that can be effectively addressed through systematic optimization logic. By utilizing Fantasy Baseball as a data-rich testbed, we have evaluated the balance between theoretical

Integer Programming and real-time decision support. Based on our evaluations, we draw the following conclusions:

1. **Bridging Optimality and Scalability:** While Integer Programming (IP) provides a rigorous benchmark, its exponential worst-case complexity makes it impractical for large-scale live environments. Our proposed *Opportunity Cost Greedy (OCG)* algorithm reduces the decision process to near-linear operations ($O(n \log n + r \cdot p)$). In standard league sizes, OCG maintains sub-second latency, processing approximately 50 million decision variables in under 25 seconds while remaining within a roughly 1–2% gap of the IP benchmark in extreme stress tests.
2. **Resilience against “Value Cliffs”:** The strategic core of OCG is its ability to quantify the **cost-of-delay**. In environments characterized by star-heavy distributions or high demand, myopic strategies often ignore the impending depletion of specific positions. OCG identifies these “Value Cliff” thresholds by looking ahead to the next pick, prioritizing scarce assets whose utility is expected to decline sharply. This robust look-ahead decision policy results in significant utility gains, particularly in scenarios with limited positional flexibility.
3. **Robustness in Competitive Ecosystems:** Multi-agent simulations indicate that OCG remains effective when competing against multiple DG agents for a shared player pool. In 2026 real-data simulations, the OCG agent consistently ranked first across all draft positions in the Yahoo setting, and remained in either first or second place throughout the draft order in the FanGraphs setting. This suggests that delay-cost logic translates into a tangible strategic advantage in realistic, multi-agent environments.

Future Extensions: Future research could transition this deterministic framework toward **Stochastic Opponent Modeling**. Our current setting still simplifies many aspects of the real-world environment. By incorporating probability distributions of opponent behavior biases instead of fixed ADP thresholds, the model could calculate more precise expected opportunity costs.

Furthermore, integrating **Risk-Aware Utility** via Monte Carlo simulations would account for player injury risks and performance volatility. From a broader perspective, the underlying logic is highly generalizable beyond sports; it provides a foundational framework for any **sequential resource procurement** or **talent acquisition** scenario where multiple agents must secure scarce, heterogeneous assets under real-time pressure and competitive depletion.

References

- [1] **FantasyPros.** *2026 MLB Player Projected Statistics (Batting/Pitching)*. <https://www.fantasypros.com/mlb/projections/>
- [2] **FantasyPros.** *2026 Fantasy Baseball Average Draft Position (ADP) Consensus*. <https://www.fantasypros.com/mlb/adp/overall.php>
- [3] **Yahoo Sports.** *Default League Settings and Head-to-Head Points Scoring System*. <https://help.yahoo.com/kb/default-league-settings-fantasy-baseball-sln6785.html>
- [4] **FanGraphs / Justin Merry.** *Ottoneu Scoring and Linear Weights for Sabermetric Evaluation*. <https://ottoneu.fangraphs.com/support>